

Relatório de Integração — Middleware Orientado a Mensagens

Projeto: BarberTime — agendamento em barbearia

Disciplina: LDAMD — PUC Minas

Sprint: 2 — Integração com MOM

Aluno: Caio Kodato

Escolha da ferramenta

Optei pelo **RabbitMQ** por ser o MOM indicado no enunciado do Projeto Integrador, com suporte a filas duráveis, roteamento por tópicos e painel de gestão. A execução local utiliza **Docker Compose** (`docker-compose.yml`), com AMQP na porta **5673** e interface de administração na **15673**, evitando conflito com outra instância RabbitMQ já presente na máquina (porta 5672).

Padrão arquitetural

Adotei **Event-Driven Architecture (EDA)** com padrão **Publish/Subscribe** via exchange **topic** (`barbertime.events`):

Elemento	Papel
API REST	Produtor — publica após gravar no PostgreSQL
RabbitMQ	Broker — roteia mensagens por routing key
Notification Worker	Consumidor — processo separado, sem HTTP entre API e worker

A comunicação segue o padrão *Message Channel* e *Event Message* (Hohpe & Woolf, *Enterprise Integration Patterns*).

Decisões de design

Decisão	Justificativa
Exchange topic	Permite novos eventos (ex.: cancelamento) sem alterar produtores existentes
Worker em processo separado	Demonstra assincronicidade real; base para apps Flutter (Sprints 3–4)
Publicação fire-and-forget	Falha no broker não reverte o agendamento já persistido
Notificação simulada em log	Sprint 2 exige consumidor funcional; push/e-mail nas próximas sprints
Variável <code>RABBITMQ_ENABLED</code>	Permite desenvolver sem broker quando necessário

Desafios encontrados

1. **Topologia compartilhada:** API e worker devem declarar a mesma exchange, filas e bindings — centralizado em `RabbitMQClient` .
2. **Conflito de porta:** outro RabbitMQ na 5672 exigiu portas dedicadas (5673 / 15673) para o BarberTime.
3. **Vínculo barbeiro–usuário:** notificação ao barbeiro depende de `barbers.user_id` → `users.email` .

Evidência de funcionamento

Evidência	Descrição
Logs da API	<code>[RabbitMQ]</code> Evento publicado: <code>appointment.requested</code> / <code>confirmed</code>
Logs do worker	<code>[NOTIFICAÇÃO → BARBEIRO]</code> e <code>[NOTIFICAÇÃO → CLIENTE]</code>
Health check	<code>GET /health</code> com <code>rabbitmq.connected: true</code>
Painel e prints	<code>ENTREGA_SPRINT2.pdf</code> e Management UI (filas e exchange)

Próximos passos (Sprints 3–4)

Integrar os aplicativos Flutter como destinatários finais das notificações, mantendo o RabbitMQ como backbone de eventos entre backend e dispositivos móveis.